phone_app/lib/phone_app/application.ex
```
{Finch, name: PhoneApp.Finch},
{Oban, Application.fetch_env!(:phone_app, Oban)},
PhoneAppWeb.Endpoint
```

The order of entries in this list does matter. By placing Oban before our endpoint—but after our Ecto repo, PubSub, and Finch—we ensure that our async jobs have access to the processes they need. Application supervision is too advanced of a topic for this book, but know that you have complete control over when processes start in your application.

You can verify that everything is working by checking the Oban config in your application. To do this, start an IEx session:

```
$ iex -S mix
iex> Oban.config()
%Oban.Config{
  ...
}
```

Now that Oban is in our app, let's create our worker!

## Write an Oban Worker

Our SMS app has a problem when it's sending messages. Messages are not delivered instantly, so we need to keep track of the status of the message. Usually, SMS messages are delivered fairly quickly—within 10 seconds—but it's actually fairly common for SMS messages to run into issues during delivery. These issues can pop up minutes or hours after attempted delivery.

When we send a message, we'll use an async job to check the status of the message until it becomes finalized. The mock SMS server that's included with this book has a delay built in—up to 15 seconds—so we can test this flow out.

Let's start by creating a new worker module and an enqueue/1 function. Create lib/phone_app/conversations/worker/status_worker.ex and add the following code:

phone_app/lib/phone_app/conversations/worker/status_worker.ex
```
defmodule PhoneApp.Conversations.Worker.StatusWorker do
  use Oban.Worker

  alias PhoneApp.Conversations.Schema.SmsMessage

  def enqueue(%SmsMessage{} = message) do
    %{"id" => message.id}
    |> new()
    |> Oban.insert()
  end
end
```